sql@banco:~$ ./processador

Processador de Consultas

SQL → álgebra relacional → grafo otimizado → plano de execução

Banco de Dados · Projeto 2 · [nomes da equipe]

ROTEIRO · 8 CRITÉRIOS · 5,0 PONTOS

Um slide por critério de avaliação

#	Critério	Peso	Pasta
1	Interface gráfica funcional	0,5	app/, scenes/
2	Parsing e validação	1,0	hu1_...
3	Conversão para álgebra relacional	0,75	hu2_...
4	Uso da junção	0,5	hu1_... hu2_... hu4_...
5	Grafo de operadores otimizado	0,5	hu3_... hu4_...
6	Heurística de redução de tuplas	0,5	hu4_...
7	Heurística de redução de atributos	0,5	hu4_...
8	Ordem de execução	0,75	hu5_...

CRITÉRIO 1 · PESO 0,5

Interface gráfica funcional

NA TELA

- F12 → COPIAR a Consulta 2
- Colar no campo e ENTER
- PRÓXIMO ▶ até o passo 3: grafo de operadores
- Botões +, –, AJUSTAR e clique em um nó

NO CÓDIGO

arquivo:linha	o quê
main.tscn	cena com todos os nós visuais
campo_consulta.gd:11	ENTER envia, colar, histórico
main.gd:183	avança os 5 passos
main.gd:280	mostra cada passo na tela
grafo_view.gd:174	desenha o grafo

`scenes/ · app/ · hu1_entrada_validacao/ · hu3_grafo_operadores/`

CRITÉRIO 2 · PESO 1,0 · O MÉTODO DE VALIDAÇÃO

Expressão regular quebra a SQL em tokens

sql_validator.gd:47 · TOKEN_RE · lida por _tokenizar (:91)

grupo da regex	reconhece
(?<str>'[^']*')	texto: 'Aberto'
(?<num>\d+(?:\.\d+)?)	número: 100, 10.5
(?<op><=|>=|<>|!=|=|<|>)	operador
(?<punct>[(),.;*])	pontuação
(?<id>[A-Za-z_]\w*)	nome ou palavra-chave
(?<bad>\S)	qualquer outro → erro

CRITÉRIO 2 · PESO 1,0 · SINTAXE E SEMÂNTICA

Gramática confere a ordem, o catálogo confere os nomes

NA TELA · PASSO 1

- Consulta 2 → ✓ tabelas e atributos
- SELECT * FROM Clientes → seta ^ no erro
- SELECT Preco FROM Cliente → atributo fora da consulta
- select cliente.nome from CLIENTE → válida

NO CÓDIGO

arquivo:linha	o quê
sql_validator.gd:7	gramática aceita
sql_validator.gd:144	SELECT → FROM → JOIN → WHERE
sql_validator.gd:253	condições com AND
sql_validator.gd:266	operadores e parênteses
sql_validator.gd:214	a tabela existe?
sql_validator.gd:311	o atributo existe? é ambíguo?
db_schema.gd:11	tabelas da Imagem 01

`hu1_entrada_validacao/`

CRITÉRIO 3 · PESO 0,75

Conversão para álgebra relacional

NA TELA · PASSO 2

- Notação linear com π, σ e ⋈
- Árvore de operadores em texto
- Condições do ON e do WHERE iguais às da SQL
- AND aparece como ∧

NO CÓDIGO

arquivo:linha	o quê
algebra_converter.gd:18	AST → árvore
algebra_converter.gd:20	cada JOIN vira ⋈
algebra_converter.gd:27	WHERE vira σ
algebra_converter.gd:30	SELECT vira π no topo
algebra_renderer.gd:56	escreve a notação
algebra_renderer.gd:133	escreve as condições

hu2_algebra_relacional/

CRITÉRIO 4 · PESO 0,5

Codificação e uso da junção

NA TELA

- Passo 3: nós ⋈ no grafo canônico
- Consulta 4: 7 JOINs aceitos
- Passo 5, H3: ⚠ produto cartesiano
- Passo 5, H4: × desaparece

NO CÓDIGO

arquivo:linha	o quê
sql_validator.gd:156	laço de N JOINs com ON
algebra_converter.gd:20	⋈ na álgebra
otimizador.gd:191	monta ⋈ com condição ou ×
otimizador.gd:228	H3: ordem pela estimativa
otimizador.gd:240	H4: só tabelas ligadas

`hu1_entrada_validacao/` · `hu2_algebra_relacional/` · `hu4_otimizacao/`

CRITÉRIO 5 · PESO 0,5

Exibição do grafo de operadores otimizado

NA TELA · PASSO 4

- Grafo otimizado: raiz π, folhas tabelas
- Custo e volume: canônica → otimizada
- Lista H1 a H4 ao lado
- Clique numa folha: ficha do nó

NO CÓDIGO

arquivo:linha	o quê
otimizador.gd:31	monta a árvore otimizada
grafo_operadores.gd:26	grafo em memória
grafo_operadores.gd:63	arestas: fluxo de resultados
grafo_operadores.gd:65	folhas = tabelas
grafo_view.gd:101	layout sem setas cruzadas
grafo_view.gd:174	desenha nós e setas

`hu3_grafo_operadores/ · hu4_otimizacao/`

CRITÉRIO 6 · PESO 0,5 · H1

Heurística de redução de tuplas

NA TELA · PASSO 5, ETAPA H1

- σ desce até a tabela (◆ realçado)
- σ{Status.Descricao = 'Aberto'} → Status (≈1000 → ≈100)
- Consulta 4: custo 7 001 000 → 26 267

NO CÓDIGO

arquivo:linha	o quê
otimizador.gd:36	bloco da H1
otimizador.gd:129	condição de uma tabela = seleção
otimizador.gd:167	σ logo acima da tabela
estimativa.gd:14	seletividades: = 1/10, intervalo 1/3
estimativa.gd:67	tuplas estimadas por nó
estimativa.gd:90	custo = tuplas processadas

`hu4_otimizacao/`

CRITÉRIO 7 · PESO 0,5 · H2

Heurística de redução de atributos

NA TELA · PASSO 5, ETAPA H2

- π logo após cada σ (◆ realçado)
- Endereco: 1 de 11 atributos
- Volume 13 792 → 8 294 (−39,9%)
- Custo sobe +5,8%: cada π lê a entrada

NO CÓDIGO

arquivo:linha	o quê
otimizador.gd:54	bloco da H2
otimizador.gd:167	π acima do σ de cada tabela
otimizador.gd:295	atributos que ainda serão usados
estimativa.gd:104	volume = tuplas × atributos

`hu4_otimizacao/`

CRITÉRIO 8 · PESO 0,75

Ordem de execução apresentada

NA TELA · PASSO 5, ÚLTIMA ETAPA

```
1. R1 ← Cliente
2. R2 ← σ{…}(R1)
3. R3 ← π{…}(R2)
…
10. RESULTADO ← π{…}(R9)
```

Rn = resultado do nó #n do grafo otimizado

NO CÓDIGO

arquivo:linha	o quê
plano.gd:27	gera o plano em pós-ordem
plano.gd:45	expressão de cada passo
plano.gd:65	cada passo usa só resultados anteriores
main.gd:224	etapas: canônica → H1…H4 → plano
main.gd:449	lista numerada na tela

hu5_plano_execucao/ · app/

PERGUNTAS PROVÁVEIS

Respostas curtas para a banca

Por que 1000 tuplas?

Não há dados reais: é a suposição padrão. Importa a comparação entre os planos.

Custo ou volume?

Custo conta linhas processadas. Volume conta linhas × colunas nas junções.

Por que a H3 encarece?

Ordenar só pelo tamanho criou produto cartesiano. A H4 corrige.

Aceita OR ou alias?

Não: o enunciado pede só = > < <= >= <> AND e parênteses.

```
sql@banco:~$ godot --headless --script res://tests/hu4/test_otimizador.gd
```

Obrigado